
프로세스

➤ 프로세스(Process)

시스템에서 실행 중인 프로그램을 프로세스라고 한다. 프로세스는 메모리 일정 구역을 할당 받아 실행되며 이렇게 할당 받은 메모리 영역을 프로텍션 도메인(Protection domain)이라고 한다.

- 프로세스는 pid에 의해 구별된다.
- 프로세스는 tree 구조를 이루며 최상위 프로세스는 systemd(init)이다.
- 프로세스는 ppid로 구별되는 상위(부모 프로세스)를 갖는다.
- fork, exec

➤ JOB

job은 현재 터미널에서 실행한 프로세스로 shell에 의해서 job 번호가 부여된다.

➤ 관련 명령어

- pstree, ps, kill, pkill, jobs, nohup, exec, top, nice, renice

```
# pstree
```

```
systemd--NetworkManager--2*[{NetworkManager}]
      |--VGAuthService
      |--agetty
      |--crond
      |--dbus-daemon
      |--gssproxy--5*[{gssproxy}]
      |--polkitd--6*[{polkitd}]
      |--rsyslogd--2*[{rsyslogd}]
      |--sshd--sshd--bash--man--less
            |--sshd--bash--pstree
      |--systemd-journal
      |--systemd-logind
      |--systemd-udevd
      |--vmtoolsd--2*[{vmtoolsd}]
```

➤ system

- RedHat에서 개발
- PID 1 프로세스: `/lib/systemd/systemd`
- 유닛 설정 파일: `/etc/systemd/system/` 또는 `/lib/systemd/system/`
- 로그 정보 관리 : `journalctl(journal)`
- `systemctl` 명령어로 관리

➤ init

- 전통적인 리눅스 초기화 시스템으로, 대부분의 유닉스 및 초기 리눅스 배포판에서 사용됨.
- `/etc/inittab` 파일을 통해 런레벨(runlevel)을 정의
- `/etc/rc.d/` 또는 `/etc/init.d/`에 위치한 스크립트로 서비스 실행.
- 런레벨 기반 작동 (0~6): 예) 3은 멀티유저 텍스트 모드, 5는 GUI.
- 로그정보 관리 : `syslog`

➤ PID 0

- swapper/sched
- 시스템 부팅 직후 가장 먼저 실행되는 커널 태스크
- CPU의 아이들(idle) 루프를 담당
- 커널이 부팅되며 생성한 최초의 태스크 (idle task)
- 사용자 접근 불가, 일반적으로 ps, top, htop에서는 보이지 않음
- 전통적으로는 swapper라고 불림
- 현대 리눅스 커널에서는 sched 또는 idle로도 불림.
- CPU 코어가 여러 개인 시스템에서는 각 코어마다 idle task가 별도로 존재하며, 모두 PID 0으로 간주됨

➤ Foreground

현재 터미널에서 제어되는 프로세스이다. 터미널은 입출력은 foreground 프로세스를 위해서 대기한다.

➤ Background

터미널과 무관하게 메모리에서 실행되는 프로그램이다. 터미널 입출력에 대해서 투명하다. 보통 daemon(service) 프로세스들이 background 에서 실행된다.

➤ ps

프로세스를 확인한다.

옵션

- a(x) : 현재 터미널에서 실행중인 프로세스
- e : 모든 프로세스
- f : 자세한 내용
- p : 특정 PID의 프로세스를 지정한다.

```
# ps -f
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	771	769	0	09:13	pts/1	00:00:00	-bash
root	787	771	0	09:13	pts/1	00:00:00	ps -f

- UID : 프로세스 실행 계정
- PID : pid
- PPID : 부모프로세스 pid
- C : idle time
- STIME : start time
- TTY : 터미널
- TIME : CPU 사용 시간
- CMD : 실행 명령


```
# ps -ef
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0	09:09	?	00:00:00	/usr/lib/systemd/systemd --switched-root --sy
root	2	0	0	09:09	?	00:00:00	[kthreadd]
root	4	2	0	09:09	?	00:00:00	[kworker/0:0H]
root	5	2	0	09:09	?	00:00:00	[kworker/u256:0]
root	6	2	0	09:09	?	00:00:00	[ksoftirqd/0]
root	7	2	0	09:09	?	00:00:00	[migration/0]
root	8	2	0	09:09	?	00:00:00	[rcu_bh]
root	9	2	0	09:09	?	00:00:00	[rcu_sched]
root	10	2	0	09:09	?	00:00:00	[lru-add-drain]
root	11	2	0	09:09	?	00:00:00	[watchdog/0]
root	13	2	0	09:09	?	00:00:00	[kdevtmpfs]
root	14	2	0	09:09	?	00:00:00	[netns]
root	15	2	0	09:09	?	00:00:00	[khungtaskd]
root	16	2	0	09:09	?	00:00:00	[writeback]
.....							
.....							

➤ kill

사용자나 프로그램이 프로세스를 제어하기 위해 시그널(Signal)을 발생시키는데 이런 시그널을 직접 보내는 명령어다. 보통 프로세스를 강제 종료할 때 사용한다.

```
# kill [-signal] [PID]
```

옵션

-1 : signal 목록(9 : 프로세스 강제 종료)

kill -l

1) SIGHUP	2) SIGINT	3) SIGQUIT	4) SIGILL	5) SIGTRAP	6) SIGABRT
7) SIGBUS	8) SIGFPE	9) SIGKILL	10) SIGUSR1	11) SIGSEGV	12) SIGUSR2
13) SIGPIPE	14) SIGALRM	15) SIGTERM	16) SIGSTKFLT	17) SIGCHLD	18) SIGCONT
19) SIGSTOP	20) SIGTSTP	21) SIGTTIN	22) SIGTTOU	23) SIGURG	24) SIGXCPU
25) SIGXFSZ	26) SIGVTALRM	27) SIGPROF	28) SIGWINCH	29) SIGIO	30) SIGPWR
31) SIGSYS	34) SIGRTMIN	35) SIGRTMIN+1	36) SIGRTMIN+2	37) SIGRTMIN+3	38) SIGRTMIN+4
39) SIGRTMIN+5	40) SIGRTMIN+6	41) SIGRTMIN+7	42) SIGRTMIN+8	43) SIGRTMIN+9	44) SIGRTMIN+10
45) SIGRTMIN+11	46) SIGRTMIN+12	47) SIGRTMIN+13	48) SIGRTMIN+14	49) SIGRTMIN+15	50) SIGRTMAX-14
51) SIGRTMAX-13	52) SIGRTMAX-12	53) SIGRTMAX-11	54) SIGRTMAX-10	55) SIGRTMAX-9	56) SIGRTMAX-8
57) SIGRTMAX-7	58) SIGRTMAX-6	59) SIGRTMAX-5	60) SIGRTMAX-4	61) SIGRTMAX-3	62) SIGRTMAX-2
63) SIGRTMAX-1	64) SIGRTMAX				

- `signal`에 대한 정의는 `/usr/local/signal`에 정의되어 있다.
- UNIX/LINUX 시스템 별로 기본적인 `Signal((SIGKILL(9), SIGSEGV(11), SIGTERM(15) ..))`을 제외하고 기타 `Signal` 번호는 OS 환경에 설치된 `signal.h`에 `define` 선언 되어 있는 것을 참고한다.

➤ nohup

프로세스 중단(hangup)을 무시하고 명령을 계속 실행한다. 이로 인해 로그아웃 이후에도 명령은 계속 실행되지만 명령을 background(&)로 돌리는 것은 아니다. 또한 표준 출력을 nohup.out 으로 변경한다.

작업 시간이 긴 명령어를 실행할 때 로그 아웃이나 터미널 다운에 대비할 수 있다.

```
# nohup [command]
```

```
# echo security class
security class

# nohup echo security class
nohup: ignoring input and appending output to `nohup.out'
# cat nohup.out
security class
```